



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

PROGRAMACIÓN ORIENTADA A OBJETOS



ASIGNATURA:

Programación Orientada a Objetos

PROFESOR:

Ing. Yadira Franco

PERÍODO ACADÉMICO:

2026-A

FECHA DE ENTREGA

22-07-2026

INFORME FINAL

TÍTULO:

Aplicación gestora de una licorería



ESTUDIANTE:

Chasi Rochina Alexis Alexander

Estrada Velazco Anderson José

Índice

1.	Introducción.....	2
2.	Propósito de la práctica.....	2
3.	Objetivos.....	2
3.1.	Objetivo General.....	2
3.2.	Objetivos Específicos	2
4.	Desarrollo	2
1.1	4.1. Temática Elegida	2
4.2.	Arquitectura del proyecto	2
4.3.	UML	3
4.4.	Pilares de POO	3
4.5.	Roles	4
4.6.	Pantallas.....	4
4.7.	Base de Datos	4
4.8.	Validaciones.....	6
4.9.	GitHub	6
4.10.	Ejecutable	7
5.	Capturas del sistema	8
6.	Conclusiones.....	9
7.	Recomendaciones	9
8.	Referencias Bibliográficas.....	10

1. Introducción

El presente informe muestra el desarrollo de una aplicación de escritorio para la gestión de una licorería “Don Liquox”, el cual cuenta con módulos para el funcionamiento eficiente del negocio, tales como el control de inventarios, gestión de clientes y usuarios, registro de ventas y un generador de reportes.

El sistema fue desarrollado en java para el manejo de la lógica, JavaFX para el diseño de la interfaz gráfica, PostgreSQL y Neon para el almacenamiento de los datos.

2. Propósito de la práctica

La práctica tiene como propósito reforzar y consolidar los conceptos clave de la programación orientada a objetos. A través de la implementación de conceptos como el encapsulamiento, herencia, polimorfismos y abstracción. Además de incorporar el patrón arquitectónica Modelo-Vista-Controlador (MVC).

3. Objetivos

3.1. *Objetivo General*

Desarrollar una aplicación para la gestión de una licorería “Don Liquox” en Java y PostgreSQL, estructurada bajo el control de roles de usuario.

3.2. *Objetivos Específicos*

- Aplicar los fundamentos de Programación Orientada a Objetos para estructurar la jerarquía de las clases y la lógica del sistema.
- Implementar el patrón de arquitectura MVC para el desarrollo de la aplicación.
- Controlar la integridad de los datos registrados por medio de validaciones.
- Diseñar interfaces dinámicas y reutilizables según el rol seleccionado.
- Desarrollar operaciones CRUD completa para cada uno de los roles definidos.

4. Desarrollo

4.1. *Temática Elegida*

El sistema tiene como temática una licorería, el problema que busca resolver el mal manejo de los inventarios, la falta de control de los clientes registrados, y he incoherencias en el registro de ventas para ello se desarrolla este sistema para mejorar el manejo del negocio.

El sistema cuenta con varios módulos según el rol ingresado (Administrador, Cajero, Reportes), el objetivo principal del sistema es manejar los distintos módulos para gestionar el negocio. La interacción con cada uno de los módulos permite gestionar los clientes y usuarios, registrar las ventas del producto y generar reportes de las ventas realizadas.

4.2. *Arquitectura del proyecto*

MCV es un patrón arquitectónico que permite separar la lógica interna del sistema con lo que el usuario interactúa y separar las responsabilidades por medio de capas.

Paquetes	Responsabilidad
----------	-----------------

App	Lugar donde se ejecuta el sistema
Model	Modelos para la creación de los objetos con sus respectivos atributos.
Controller	Punto donde se conectan las vistas FXML con la lógica del negocio según las acciones del usuario.
Dao	Implementa de la lógica necesaria para el acceso de los datos y comunicación SQL de la base de datos de PostgreSQL.
db	Estable la conexión de la base de datos y el script de la creación base de datos.
view	Archivos FXML donde se define el diseño de las interfaces gráficas.

4.3 UML

Para representar de forma gráfica la estructura del proyecto se realizó un UML desarrollado en la plataforma draw.io, donde se detalla el manejo de clases, atributos, métodos, herencias e interfaces.

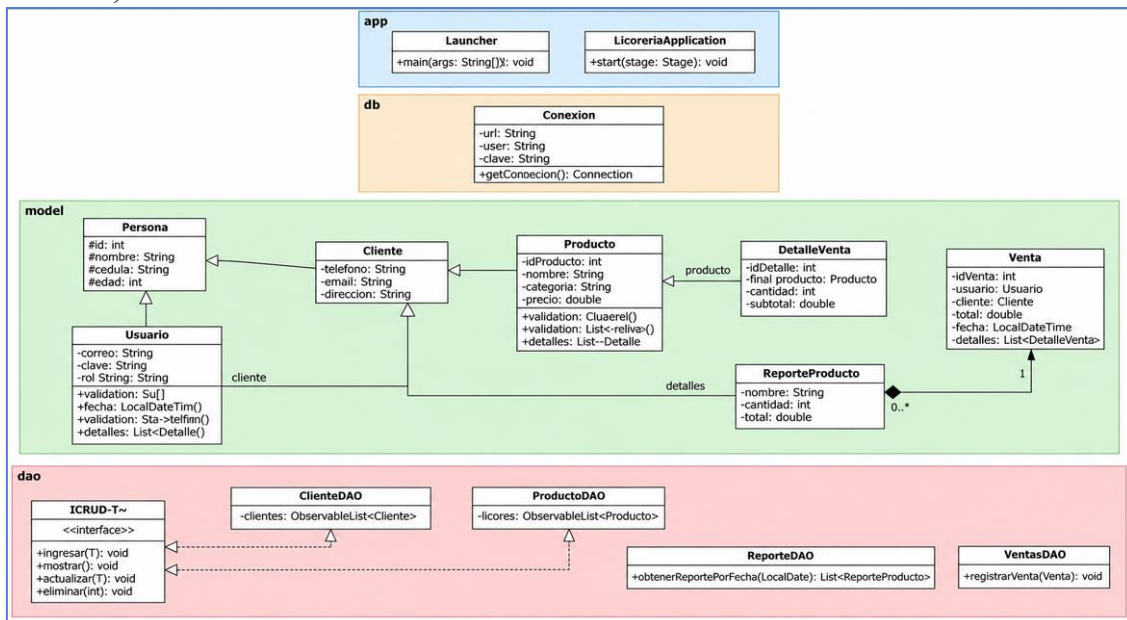


Figura1. Diagrama UML del sistema

4.3 Pilares de POO

Para el desarrollo de la aplicación es fundamental el uso de los pilares de programación orientada a objetos.

Encapsulamiento: Los atributos de las clases modelos (persona, usuario, productos, etc) son declaradas como privada. El acceso y modificación de estos es necesario de los métodos gets y sets, con el fin de garantizar el control de los datos.

Herencia: La clase Persona se define como abstracta ya comparten atributos con las demás clases. Las clases hijas Usuario y Cliente son una extensión de Persona, lo que permite heredar su estructura y añadir atributos propios.

Abstracción: La interfaz ICRUD<T> establece que las clases que usen esta interfaz estén obligadas a utilizar las funciones que se establecen (registrar, listar, actualizar, eliminar). Cada DAO implementa esta lógica para los distintos SQLs que usa.

4.4 Roles

El sistema está estructurado por roles donde se habilitará las opciones según el rol en que se encuentre.

Administrador: Gestiona el inventario, usuarios, clientes del sistema, registro de ventas y generación de reportes.

Cajero: Genera las ventas en base a los clientes registrados.

Reportes: Genera de reportes a Excel de los productos mas vendidos, y de los detalles de una venta.

4.5 Pantallas

La estructura de las pantallas se encuentra realizadas por archivos FXML en las cuales están se asocian a un controlador para su uso dentro del sistema.

Pantalla Módulo	Archivo FXML	Controlador	Propósito
Inicio de Sesión	login.fxml	LoginController.java	Interfaz de autenticación de credenciales y acceso al sistema
Dashboard	dashboard.fxml	DashboardController.java	Contiene el menú de forma dinámica que se adapta al rol activo, ocultando o mostrando opciones.
Gestión de Productos	Productos.fxml	ProductoController.java	Permite administrar el catálogo de bebidas, precios unitarios y operaciones CRUD.
Gestión de Clientes	Cliente.fxml	ClienteController.java	Apartado dedicado registro, actualización y consulta de los clientes.
Gestión de Usuarios	Usuarios.fxml	UsuarioController.java	Módulo para la modificación o eliminación de los usuarios ya existentes.
Registro de Ventas	Ventas.fxml	VentaController.java	Interfaz de facturación donde se selecciona al cliente, se añaden productos al carrito, se calculan subtotales y se registra la transacción.
Módulo de Reportes	Reportes.fxml	ReportesController.java	Pantalla de análisis filtrar el historial de ventas por fechas y exportar estadísticas de los productos más vendidos.

4.6 Base de Datos

La persistencia de la información del sistema “Don Liquox” se encuentra implementada en PostgreSQL, aprovechando su robustez y conectividad mediante JDBC. Para garantizar la disponibilidad la base de datos está alojada en la nube utilizando la plataforma Neon.

La base fue desarrollada bajo las necesidades del negocio, separando la parte de ventas con respecto a las demás debido al modelo en que se trabaja.

A. Estructura de Tablas y Atributos

1. Tabla usuarios

- **Propósito:** Almacena la información del personal autorizado para operar el sistema, vinculando sus datos personales con sus credenciales de seguridad.
- **Campos:** Id, nombres, cédula, edad, correo electrónico, contraseña de acceso y rol.

2. Tabla clientes

- **Propósito:** Registro de clientes de la licorería para asociarlos al módulo de ventas.
- **Campos:** Id, número de cédula, nombres, edad, teléfono, correo electrónico y dirección física.

3. Tabla productos

- **Propósito:** Gestiona el inventario de bebidas disponibles.
- **Campos:** Id del producto, nombre comercial, categoría (ej. cervezas, vinos, etc.), precio de venta y manejo de stock.

4. Tabla ventas

- **Propósito:** Funciona como la cabecera transaccional que agrupa la información global de una compra realizada en el local.
- **Campos:** Id de la venta, LocalDateTime de la transacción, total, y enlaces de relación hacia el usuario y el cliente.

5. Tabla detalle_venta

- **Propósito:** Permite visualizar las ventas en una determinada fecha.
- **Campos principales:** Id del detalle, referencia a la venta general, referencia al producto adquirido, cantidad de unidades vendidas y el subtotal calculado.

B. Claves Primarias y Claves Foráneas

El diseño relacional implementa restricciones a nivel de esquema para garantizar la integridad referencial, evitando la existencia de registros inconsistentes.

- **Claves Primarias (PK):** Cada tabla posee un identificador unívoco de tipo entero (ej. id_usuario, id_cliente, etc) configurado como clave primaria auto-incremental (SERIAL o IDENTITY en PostgreSQL). Lo que garantiza la existencia única de los datos registrados.

- **Claves Foráneas (FK):** Establecen las relacionales y dependencia entre las tablas:
 - La tabla ventas contiene dos claves foráneas: “*id_usuario*” que vinculada a la tabla usuarios para identificar qué empleado efectuó la venta y el “*id_cliente*” que se vincula a la tabla clientes.
 - La tabla “*detalle_venta*” contiene claves foráneas hacia “*id_venta*” para asociar a la factura y hacia “*id_producto*” para descontar y consultar el catálogo de inventario.

C. Datos de Prueba y Poblado Inicial

Para verificar el correcto funcionamiento de la base de datos (operaciones CRUD, la conectividad DAO), se realizó un script inicial:

- **Usuarios de Prueba:** Se insertaron perfiles quemados con credenciales para evaluar de forma el control de acceso por roles:
 - Administrador: admin@admin.com
 - Cajero: cajero@cajero.com
 - Reportes: reportes@cajero.com
- **Catálogo Inicial de Productos:** Se registraron las bebidas más populares y datos aleatorios para validar el registro de inventario.
- **Cientes Frecuentes:** Registro de los clientes frecuentes para garantizar una base sólida y la validaciones de los campos.
- **Transacciones Históricas simuladas:** Inserción de ventas y detalles de prueba que permiten comprobar los cálculos del DAO de reportes al generar archivos exportables con los productos más vendidos.

4.7 Validaciones

Para garantizar el correcto manejo de los datos se usaron las excepciones para evitar que el programa se cierre por errores de ingreso y se generaron ventanas de alerta para los usuarios del sistema donde se detalle el error que han cometido.

Las validaciones incorporadas en el sistema son: validación de credenciales, campos vacíos, error de registros (números o letras) en donde no corresponda y errores de conexión a la base de datos.

4.8 GitHub

Para el desarrollo colectivo del proyecto se utilizó GitHub que permitió trabajar de forma amena.

El repositorio creado de la aplicación se encuentra estructurado de la siguiente manera: LicoreriaApp/

```
|---- src/main/java/com/DonLiquox/licoreria/
|    |-- app/ (Launcher.java, Main.java, LicoreriaApplication.java)
|    |-- model/ (Persona.java, Usuario.java, Cliente.java, Producto.java, Venta.java,
```

```

DetalleVenta.java)
|--- dao/ (ICRUD.java, UsuarioDAO.java, ProductoDAO.java, ClienteDAO.java,
VentasDAO.java, ReporteDAO.java)
|---controller/ (LoginController.java, DashboardController.java,
ProductoController.java, etc.)
|----- db/ (Conexion.java)
|---- src/main/resources/
|    |--- com/DonLiquox/licoreria/view/ (login.fxml, dashboard.fxml, Productos.fxml,
etc.)
|    | |----- css/ (estilos.css)
|    |----- pom.xml

```

Cada archivo se encuentra organizado bajo el parámetro establecido de la arquitectura MVC.

Para los avances progresivos del proyecto se los realizaba por medio de comits, que garantizaba un manejo optimo.

Los comandos que usaron fueron: git pull (descargara la versión más actual), git add . (prepara los archivos para ser subidos=, git commit "comentario"(comentario de los cambios realizados) y git push (sube los cambios ya hechos),

4.9 Ejecutable

Para el despliegue final de la aplicación, se desarrolló un .exe que tendrá toda la lógica del programa.

Para ello se necesitan herramientas de compilación, gestión de dependencias y empaquetado de librerías gráficas de JavaFX:

Java Development Kit (JDK 26): Utilizado en el jpackage.exe, que se encarga de estructurar y empaquetar el instalador final del software.

Maven y Maven Wrapper (mvnw.cmd): Gestión de dependencias, el cual permitió establecer el proceso de construcción (build) sin necesidad de una instalación previa de Maven.

Plugin maven-shade-plugin: Empaquetado para la configuración del .jar con todas sus dependencias en un único archivo ejecutable consolidado de tipo *fat jar* (LicoreriaApp-1.0-SNAPSHOT.jar).

Plugin jpackage-maven-plugin: Extensión de Maven que automatiza la invocación de jpackage durante la fase de empaquetado del proyecto.

WiX Toolset (v3.11.2): Utilizada por el jpackage para procesar y construir el paquete instalador final en formato ejecutable (.exe) en Windows.

5 Capturas del sistema

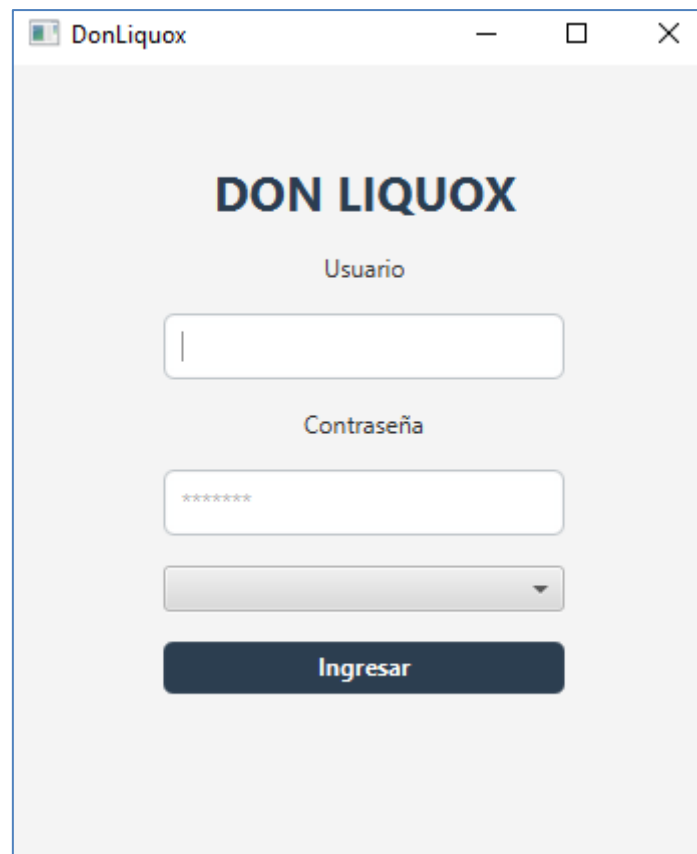
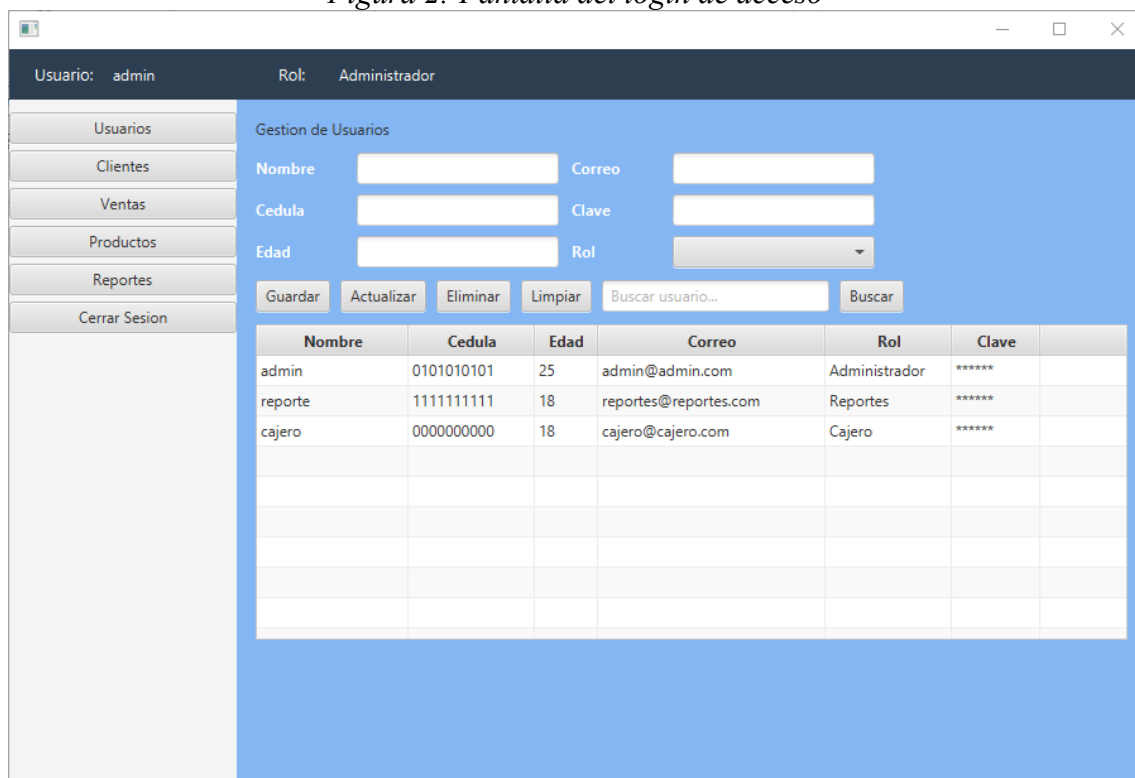


Figura 2.-Pantalla del login de acceso



Nombre	Cedula	Edad	Correo	Rol	Clave	
admin	0101010101	25	admin@admin.com	Administrador	*****	
reporte	1111111111	18	reportes@reportes.com	Reportes	*****	
cajero	0000000000	18	cajero@cajero.com	Cajero	*****	

Figura 3.-Pantallas Dashboard y Gestión de Usuarios

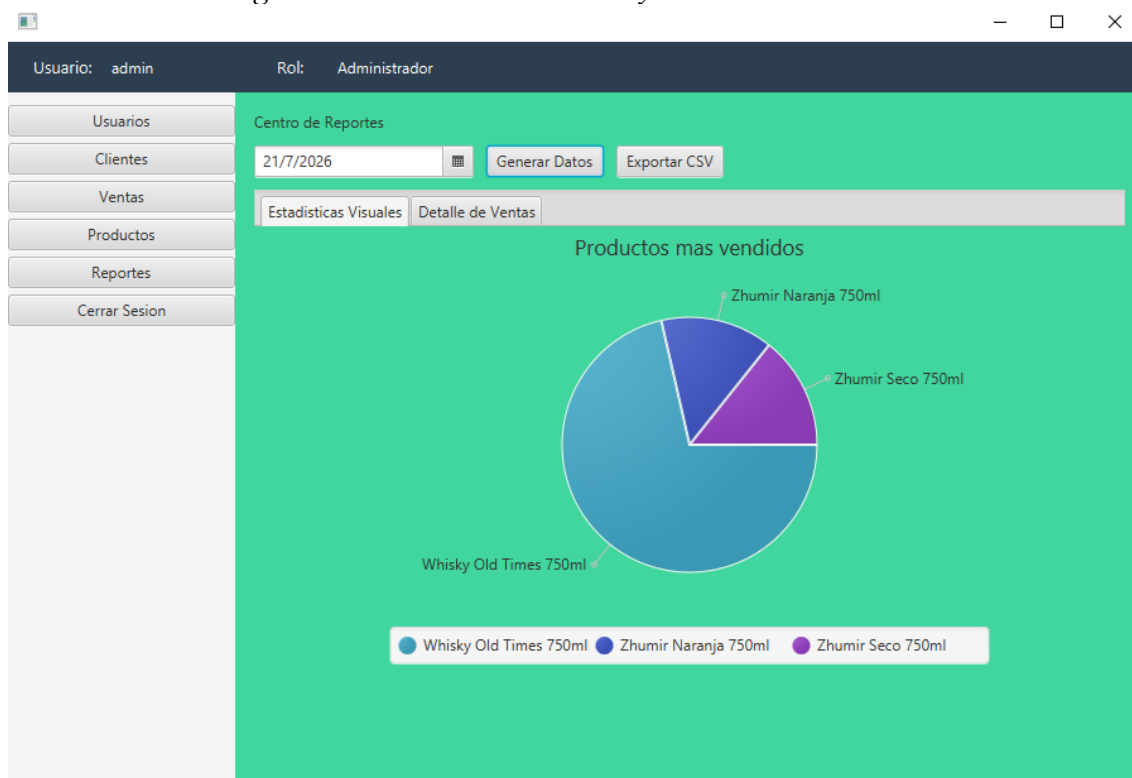


Figura 4.- Pantalla Estadísticas de producto mas vendidos

6. Conclusiones

Se desarrollo con exito la aplicación de escritorio para la licorería “Don Liquox”, integrando la lógica en Java, la interfaz dinámica en JavaFX y la persistencia en PostgreSQL.

La implementación de los pilares de la Programación Orientada a Objetos y del patrón arquitectónico MVC permitió estructurar un código modular, mantenible y escalable.

La incorporación de validaciones robustas y un control de roles de usuario garantiza el uso seguro de los módulos operativos.

7. Recomendaciones

Se recomienda implementar en futuras versiones encriptación de contraseñas mediante algoritmos seguros como BCrypt para proteger las credenciales en la base de datos.

Es idóneo optimizar el módulo de reportes incorporando gráficos estadísticos interactivos para mejorar la lectura gerencial de los datos.

Se sugiere generar un módulo con detalles más específicos de cada venta y no solo limitarse a id automatizados.

8. Referencias Bibliográficas

PostgreSQL Global Development Group. (2026). *Documentación de PostgreSQL: Sistema de gestión de bases de datos relacionales*. Recuperado de: <https://www.postgresql.org/docs/>

Oracle. (2026). *Java Platform, Standard Edition: Guías de desarrollo y API*. Oracle Corporation. Recuperado de: <https://docs.oracle.com/en/java/>

Apache Software Foundation. (2026). *Apache Maven: Gestión de dependencias y automatización de compilación*. Recuperado de: <https://maven.apache.org/guides/>

Oracle. (2026). *Herramienta jpackage para la creación de instaladores nativos en Java*. Recuperado de: <https://docs.oracle.com/en/java/javase/21/docs/specs/man/jpackage.html>